

PUZZLE 2 PBE

El segon *puzzle* de l'assignatura proposat consta de programar una versió gràfica del puzzle anterior, amb la biblioteca Ruby-GNOME2/gtk3 en el meu cas que estem codificant en Ruby. Hi haurà una finestra principal formada per un Label que demana *login* amb el carnet de la universitat, i un botó Clear per a aparèixer a la pàgina principal altre cop un cop llegit el primer UID.

1. Variació del codi anterior

Prèviament, abans de seguir amb el segon puzzle, vaig fer una altra versió del primer, de manera que el codi fos menys extens. He utilitzat doncs, una altra llibreria ja que abans feia ús de la *smartcard* i ara, la *rubby-nfc* i *logger*. A més a més, s'ha corregit la sortida de l'uid ja que imprimia més caràcters dels necessaris.

El codi en qüestió és el següent:

Dintre de la funció *read_uid*, es determina el tipus de targeta que serà llegida.

S'ha hagut d'instal·lar o bé comprovar prèviament, la llibreria *libnfc-dev* i *ruby-nfc*:

```
$ sudo apt-get install libnfc-dev (Paquet per a treballar amb RFID)
```

```
$ gem install ruby-nfc (Per a NFC)
```

La sortida per pantalla correspon doncs a:

```
pi@raspberrypi:~/Desktop/pbe $ ruby puzzle1b.rb
UID llegit: F6D67882
pi@raspberrypi:~/Desktop/pbe $
```

```
require 'ruby-nfc'
require 'logger'

class Rfid
  def initialize
    @logger = Logger.new(STDOUT)
    @logger.level = Logger::INFO
  end

  def read_uid
    NFC::Reader.all.each do |reader|
      reader.poll(Mifare::Classic::Tag) do |tag|

        if tag.class == (Mifare::Classic::Tag)
          uid = tag.uid.unpack1('H*').upcase[0,8]

          return uid
        end
      end
    end
  end

  if __FILE__ == $0
    rf = Rfid.new
    uid = rf.read_uid
    puts "UID llegit: #{uid}" if uid
  end
end
```

2. Instal·lació de ruby-gtk3 i llibries utilitzades

Per a poder seguir amb el treball, s'ha instal·lat **ruby-gtk3** amb l'ajuda d'un lloc web on especificava les comandes a escriure a la Terminal de la Raspberry:

```
$ sudo apt-get install ruby-gtk3 (Aquesta comanda instal·larà totes les dependències necessàries per a començar a treballar amb GTK3 en Ruby)
```

```
$ sudo apt-get install libgtk-3-dev (Gtk3 a nivell de sistema)
```

```
$ gem install gtk3 (Instal·lar el binding per Ruby, GTK3 està escrit en C)
```

Tot i això, vaig tenir un petit inconvenient, ja que se m'instal·lava la versió 4.2.7 de GTK, i realment volia acabar utilitzant la GTK3 per assegurar-me de no tenir problemes amb el meu perifèric ACR122U per incompatibilitats. Per tant, es va haver d'especificar la versió:

```
$ gem install gtk3 -v 3.4.5
```

Altres llibreries que seran utilitzades són, principalment, el fitxer del puzzle1.rb, que serà com una biblioteca per aquest nou codi.

3. Codi principal

Per escriure aquest programa utilitzant la biblioteca gràfica determinada anteriorment, s'ha hagut de fer una búsqueda d'exemples de codis per així entendre bé el seu funcionament i finalitat. A pàgines webs de *github* s'han trobat aquest tutorials.

A més a més, s'ha hagut de tenir en compte que, el mètode *read_uid* del primer puzle és bloquejant de manera que s'ha hagut de considerar la creació d'un nou *Thread* auxiliar.

El codi crat ha estat doncs:

```
require 'gtk3'
require_relative 'puzzle1b'

app = Gtk::Application.new("org.gtk.example", :flags_none)

rf = Rfid.new # classe del puzzle1b

app.signal_connect "activate" do |application|
  window = Gtk::ApplicationWindow.new(application)
  window.set_title("rfid_gtk")
  window.set_default_size(200, 150)
  window.set_border_width(10)

  css_provider = Gtk::CssProvider.new
  css_provider.load_from_path(File.expand_path("style.css", __dir__))
  Gtk::StyleContext.add_provider_for_screen(Gdk::Screen.default, css_provider, Gtk::StyleProvider::PRIORITY_USER)

  label = Gtk::Label.new()
  enunciat = Gtk::Box.new(:vertical, 10)
  enunciat.pack_start(label, expand: true, fill: true, padding: 5)
  button = Gtk::Button.new(label: "Clear")
  enunciat.pack_start(button, expand: false, fill: true, padding: 5)

  window.add(enunciat)

  def llegir_targeta (label, rf, window)
    label.set_text("Siusplau, identifica't amb el teu carnet de la universitat:")
    window.style_context.add_class("reading") #lila

    Thread.new do
      uid = rf.read_uid

      GLib::Idle.add do
        label.set_text("UID llegit: #{uid}")
        window.style_context.remove_class("reading")
        window.style_context.add_class("success")
        false
      end
    end
  end
end
```

```

llegir_targeta(label,rf>window)

button.signal_connect "clicked" do
  llegir_targeta(label,rf>window)
end
window.show_all
end
puts app.run([$0] + ARGV)

```

Les dues primeres línies del codi indiquen que es necessita la llibreria GTK3 per a la interfície gràfica i el fitxer puzzle1b.rb, que conté la implementació de la classe Rfid.

A continuació es crea l'aplicació on es mostrarà el contingut, i a més, es genera una instància de *Rfid*, que serà l'encarregada de llegir les targetes. També es crea la finestra principal de l'aplicació, s'assigna el seu nom ("rfid_gtk"), es defineix una mida per defecte i s'estableix un marge de 10 píxels al voltant del contingut.

Per donar estil a l'aplicació, es crea un gestor d'estils CSS, es carrega l'arxiu *style.css*, el qual serà explicat al següent punt de la memòria, i s'aplica l'estil a tots els elements de l'aplicació.

A nivell d'elements visuals, es crea un *Label* que mostrarà la informació llegida, i un contenidor vertical (*Box*) per organitzar correctament els components. A més, es crea un botó anomenat *Clear* que ens permetrà reiniciar la lectura de la targeta.

Per simplificar el codi i fer-lo més eficient, es defineix una funció ***llegir_targeta(label, rf, window)***. Aquest mètode ens canvia el text del *label* per indicar a l'usuari que s'ha d'identificar amb el carnet, activa un estil visual per indicar que s'està realitzant la lectura, i llegeix l'*uid* de la targeta en un fil separat per tal d'evitar bloquejar la interfície.

Per acabar, un cop creat el mètode, es fa una primera crida passant per paràmetres el *label*, *rf* i *window* de l'aplicació, i es determina l'acció del botó *Clear* que correspondrà al fet que quan es faci *clic*, es tornarà a cridar *llegir_targeta*, fent que el procés de lectura reiniciï. Mostrarem tots els elements de la interfície, i s'executarà l'aplicació.

4. Fitxer CSS

Aquest fitxer és on es determinar l'estil per a l'aplicació:

```

/* style.css */
window {
  background-color: #C3DA63; /*verd*/
}

label {
  font-size: 16px;
  color: black;
  font-family: 'Trebuchet MS', sans-serif;
}

button {
  background-color: lightgray;
  border: 2px solid #666;
  padding: 10px;
  font-size: 14px;
  font-family: 'Trebuchet MS', sans-serif;
  border-radius: 15px;
}

/* quan el ratolí està a sobre */
button:hover {
  background-color: #888;
  border-radius: 15px;
}

button:active {
  background-color: gray;
  border-radius: 15px;
}

/* Estil per a la finestra quan es llegeix una targeta */
window.reading {
  background-color: #B6A8FD; /*lila*/
}

```

La finestra principal (*window*) té un fons de color verd corresponent a #C3DA63.

El *label* mostra el text de color negre, amb una mida de lletra de 16px i la font “Trebuchet MS”.

El botó *Clear* té un color de fons gris clar, una vora de 2px amb cantonades arrodonides, i la mida de la lletra és de 14px. A més, quan el ratolí passa per sobre (*hover*), el color de fons es torna més fosc, i quan es prem (*active*), el color de fons es torna encara més fosc.

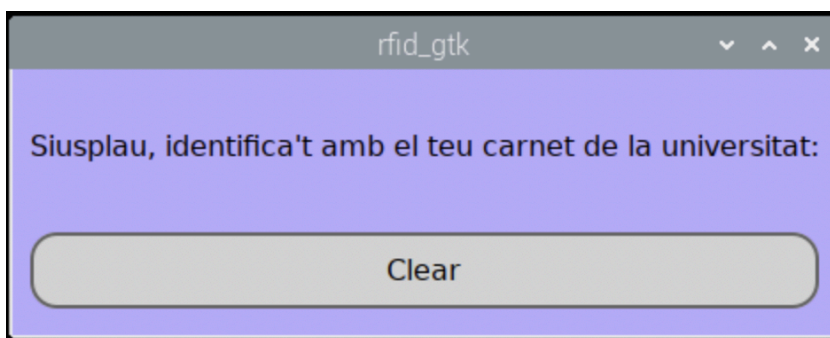
El mode de lectura de targetes (*window.reading*) canvia el color de fons de la finestra a lila mentre s'està llegint una targeta.

Per a crear aquest fitxer també s'ha hagut de fer una recerca d'exemplars i a més, per a determinar els colors, s'ha trobat una pàgina *web* que et permet escollir els tons que prefereixis i ja en allà, t'ho converteix en hexadecimal per així afegir-lo al fitxer:

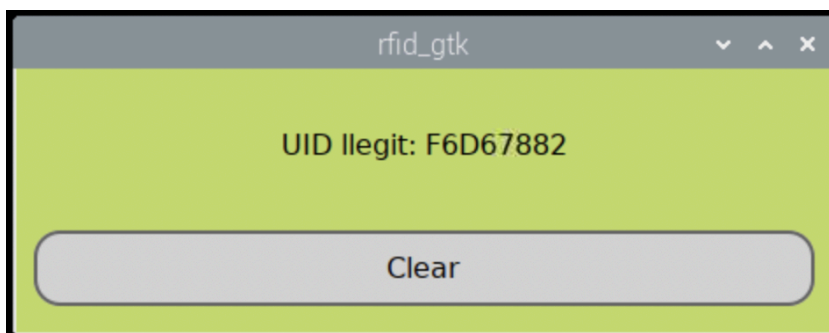
<https://www.mclibre.org/consultar/htmlcss/css/css-color.html>

5. Sortida per pantalla

Pantalla inicial:



Un cop llegit l'uid:



Si passem el ratolí per sobre el botó *Clear*:

